# Notes on *Recursive Introduction to the Theory of Computation by Carl Smith [2]*

Kedar Mhaswade

30 June 2026

**Reflection 1** (Introduction)**.** These are the author's highly personal notes based on this book [2]. They may occasionally sound like author's conversation with the reader (or even himself). Even if they are personal and appear in the public domain, they may help the reader, although the author isn't exactly sure how. He wrote them for 0) the love of mathematics and writing 1) LaTeX practice (of writing hopefully readable mathematics, albeit longer than appearing in standard texts), and 2) reliable archival (paper, on which most work is created, tends to get lost) that incurs minimal overhead.

These notes are interspersed with "reflection boxes" like this one. They are meant to express author's 'rough' thinking process (mental catharsis of sorts), of which frustration is often an inseparable part. The author writes mostly in first person for an authentic conversational tone.

## 0.1 Preface

> *Audience, level, and treatment–a description of such matters is what prefaces are supposed to be about.*
>
> — Paul Halmos
> I Want to be a Mathematician [1]

This is a graduate-level textbook that can be covered in one semester. It defines 'computation' succinctly:

> *Computation is the **movement and transformation of 'data' through 'processors'**, which are understood as expressly designed systems made of well-defined components.*
>
> — Carl Smith

The book describes three "models of computation", each offering a different perspective. It also describes which functions are *computable* (emphasis mine):

> *Given the **firm knowledge of what is and what is not computable**, we move on to consider the complexity of computing the things that are computable.*
>
> — Carl Smith

About 'recursion' in the title of the book:

> *The title of this book contains the word 'recursion,' which has several meanings, many of which are appropriate. The original meaning of the word **recurse** was "a looking back." Indeed, in this book we look back at the fundamental results that shaped the theory of computation as it is today. The object of study for most of the book is the **partial recursive functions**, functions that derive their name from the way they are defined, using an operator that 'looks back' to prior function values. There is also a **recursion theorem**, which enables us to construct programs that can look back on their own code. In this treatment, recursion theorems are introduced relatively early and used often.*
>
> — Carl Smith

**Reflection 2.** For a while now, I wanted to understand the basic theory of computation better. In particular, I wanted to understand the equivalence between Turing machines and recursive functions as "models of computation." A cursory glance at the book finally made me commit to doing it along with other things I have committed to.

The textbook is about a fundamental computation itself. It may not be relevant in the age of faulty A.I., but I don't care. This was overlooked during my previous graduate days, and I wanted to try and fill that gap. There are several excellent books on this topic, but I will take a look at them later. For now, I have this book.

I may not be able to finish it in time, but try I shall.

# Chapter 1

# Models of Computation

> *The first artifact of computation that we will dismiss is the notion that arguments may be of different types. All our inputs, outputs, temporary variables, etc., will be natural numbers (members of $\mathbb{N}$). We will argue informally that* ***all the other commonly used argument types are included solely for convenience of programming and are not essential to computation****.*
>
> — Carl Smith

## 1.1 Random Access Machines

## 1.2 Partial Recursive Functions

### Base Functions

Not all of these need be 'functions', but are defined as such to keep the notation consistent.

$$\begin{aligned} &\text{(Zero)} && Z(x) = 0 \ \forall x \\ &\text{(Successor)} && S(x) = x + 1 \ \forall x \\ &\text{(Projection or Selection)} && U_j^n(x_1, x_2, \ldots, x_j, \ldots, x_n) = x_j \end{aligned} \tag{1.1}$$

**Reflection 3** (No Identity Function!)**.** Interestingly, he doesn't define the identity function, $I(x) = x$, as a base function. However, $I = U_1^1$. His thrust may be on defining more generic (and hence more powerful) functions.

**Definition 1** (Functions defined by 'primitive' recursion)**.** A function $f$ of $n + 1$ arguments is defined by *primitive recursion* from 1) $g$, a function of $n$ arguments, and 2) $h$, a function of $n + 2$ arguments, if and only if for each[1] $x_1, x_2, \ldots, x_n$:

$$\begin{aligned} f(x_1, x_2, \ldots, x_n, 0) &= g(x_1, x_2, \ldots, x_n) \\ f(x_1, x_2, \ldots, x_n, y + 1) &= h(x_1, x_2, \ldots, x_n, y, f(x_1, x_2, \ldots, x_n, y)) \end{aligned} \tag{1.2}$$

For the $n = 0$ case (i.e., functions applied to zero arguments[2]), we adopt the convention that a function of 0 arguments is a *constant*.

[1] I think he means for every sequence of $n$ arguments, $x_1, x_2, \ldots, x_n$, not for each argument like $x_1$.

[2] Many programming languages call such functions `void`.

*The recursion of the above definition is 'primitive' because the value $f(x_1, x_2, \ldots, x_n, y)$ can be determined from the value of $f(x_1, x_2, \ldots, x_n, z)$ for some $z < y$ ("looking back"). **Forms of recursion that are not primitive will be discussed extensively later in the book**[a].*

[a] "Non-primitive' recursive functions? I am curious!

— Carl Smith

**Reflection 4** (General vs. Concrete)**.** This definition [1.2] appears too complicated at first. A typical definition of a recursive function usually refers to the hackneyed 'factorial' (factorial : $\mathbb{N} \to \mathbb{N}$) that is applied to a nonnegative integer:

$$\text{factorial}(x) = \begin{cases} 1, & \text{if } n = 0 \\ x \times \text{factorial}(x-1), & \text{otherwise} \end{cases}$$

This is typical in formal sciences, especially math and computer science. We tend to *generalize* operations. In time, we become better at finding a *level of generality* that balances flexibility with complexity. In this case, to define $f$ in terms of itself, we also define $g$ and $h$. To keep them generic, we consider functions with several arguments.

How does one *conceive* a definition such as [1]? I believe they start with some definition and refine it to a sufficient level of generality. Experience may be their guide to judge that level. With functions, their *arguments* and *return values* are all we have. Using that effectively may have also helped such a conception. $f : \mathbb{N}^{n+1} \to \mathbb{N}$ is defined by cases:

1. If the last argument is 0, $f$ returns what $g : \mathbb{N}^n \to \mathbb{N}$, applied to the remaining ($n$) arguments, returns.
2. Otherwise, $f$ returns what $h : \mathbb{N}^{n+2} \to \mathbb{N}$, applied to the remaining ones ($n$) and two carefully chosen arguments (that is, a total of $n + 2$ arguments), returns. In this case, the calculation of the last argument to $h$ requires a recursive application of $f$.

Nonetheless, it feels mysterious at first.

**Definition 2** (Functions defined by 'composition')**.** A function $f$ of $n$ arguments is defined by *composition* from 1) $g$, a function of $m$ arguments, and 2) functions $h_1, h_2, \ldots, h_m$, each of $n$ arguments, if and only if, for each $x_1, x_2, \ldots, x_n$:

$$f(x_1, x_2, \ldots, x_n) = g(h_1(x_1, x_2, \ldots, x_n), h_2(x_1, x_2, \ldots, x_n), \ldots, h_m(x_1, x_2, \ldots, x_n)) \tag{1.3}$$

The $m = 1$ case yields the traditional composition scheme, e.g., $f(x) = g(h(x))$.

**Definition 3** (Class of primitive recursive functions)**.** The class of primitive recursive functions is the smallest class of functions containing all the base functions that is closed under the operations of primitive recursion and composition.

## Examples

**Reflection 5** (Creating functions from the 'base functions')**.** The book provides a definition of a function that adds its two arguments. Starting from the definition [1] and base functions [1.1], I will try to *derive* it here. Let's denote the function as $A$ which returns an integer when applied to two integer arguments, $x, y$.

From equation [1.2], it follows that

$$\begin{aligned} A(x,0) &= g(x) \\ A(x,y+1) &= h(x,y,A(x,y)) \end{aligned} \tag{1.4}$$

Since our function $A$ applies to two arguments, **our job is to find definitions of $g$ (applies to one argument) and $h$ (applies to three arguments) so that $A$ returns the sum of its arguments**.

From the first of these equations, it is sufficiently clear that $g$ is the identity function, $g = U_1^1$, which returns its argument when applied to any argument: $g(x) = x$.

If $y \neq 0$, applying $A$ results in at least one more application of $A$. This process continues until $y$ becomes zero, when applying $A$ results in applying $g$, which returns the first argument ($x$). We say that, at that point, the (primitive) recursion 'bottoms out.' We can visualize applications of $h$ thus:

$$\begin{aligned} &1) A(x,y) = h(x,y-1,A(x,y-1)) \\ &\qquad\qquad\quad | \\ &\quad 2) h(x,y-2,A(x,y-2)) \\ &\qquad\qquad\quad | \\ &\quad 3) \ h(x,y-3,A(x,y-3)) \\ &\qquad\qquad\quad | \\ &\qquad\qquad \cdots \\ &\qquad\qquad\quad | \\ &\quad y) \ h(x,0,A(x,0)) = h(x,0,x) \end{aligned}$$

Therefore, one application of $A$ to two arguments, $x, y$, results in $y$ applications of $h$; in each application of $h$, the last argument is decremented. What should $h$ return in general then?

If $h$ returned the successor of its last argument, every application of $h$ simply counts up from $x$, $y$ times! Therefore, $h$ is just the successor of its last argument: $h(x,y,z) = S(z)$. In general, using our base functions:

$$\begin{aligned} g(x) &= x \\ h(x_1,x_2,x_3) &= S(x_3) \\ A \text{ is now defined by primitive recursion:} & \\ A(x_1,0) &= g(x_1) \\ A(x_1,x_2+1) &= h(x_1,x_2,A(x_1,x_2)) \end{aligned} \tag{1.5}$$

**Reflection 6** (Defining multiplication, $M$, by primitive recursion and composition)**.**

$$\begin{aligned} M(x_1,0) &= g(x_1) \\ M(x_1,x_2+1) &= h(x_1,x_2,M(x_1,x_2)) \end{aligned} \tag{1.6}$$

Define $g, h$.

Since the product of anything and 0 is 0, $g$ is the zero function, which we simply write as 0.

It is similar to addition, but I needed some experimentation. When the recursion bottoms out, $M$ returns what $g$ returns (i.e., 0). If $h$ returns the sum of its first and last arguments, things work out nicely.

Consider $M(2,3)$:

$$\begin{aligned} M(2,3) &= h(2,2,M(2,2)) \\ &= h(2,2,h(2,1,M(2,1))) \\ &= h(2,2,h(2,1,h(2,0,M(2,0)))) \\ &= h(2,2,h(2,1,h(2,0,0))) \\ &= h(2,2,h(2,1,2)) \\ &= h(2,2,4) \\ &= 6 \end{aligned} \tag{1.7}$$

Therefore,

$$\begin{aligned} g(x) &= 0 \\ h(x_1,x_2,x_3) &= A(x_1,x_3) \quad \text{See equation [1.5]} \\ M \text{ is now defined by primitive recursion:} & \\ M(x_1,0) &= 0 \\ M(x_1,x_2+1) &= h(x_1,x_2,M(x_1,x_2)) \end{aligned} \tag{1.8}$$

Indeed, one can start mechanically[a] to define new functions from simple, yet general (and hence powerful) base functions (See equation [1.1]), primitive recursion (See definition [1]), and composition (See definition [2])!

[a] Some creativity is also needed.

> *We can now define a very large class of functions. This class is large enough to include all the functions that we can actually compute today. In fact, all the functions we are likely to ever be able to compute are included in the class of primitive recursive functions (which is closed under primitive recursion and composition).*
>
> — Carl Smith

**Reflection 7** (The predecessor function)**.** We already have a successor function, $S$ (See equation [1.1]). Let's define a predecessor function, $P$. The only peculiarity is that the predecessor of any number less than or equal to 0 is 0. Such subtraction where a negative number is *never* produced is called "proper subtraction".

We start with primitive recursion: $P$ is applied to one argument, therefore, $n = 0$, i.e., $g$ is a 'constant' (applies to 0 arguments) and $h$ applies to 2 arguments:

$$\begin{aligned} P(0) &= 0 \\ P(x+1) &= h(x,P(x)) \end{aligned} \tag{1.9}$$

Our task is the same: Define $h$.

Applying $P$ to a familiar number like 2, we get:

$$P(2) = h(1,P(1)) = h(1,h(0,P(0))) = h(1,h(0,0))$$

This should return 1. That means, $h$ should just return the first argument. It just works!

$$P(2) = h(1,P(1)) = h(1,h(0,P(0))) = h(1,h(0,0)) = h(1,0) = 1$$

Here is how we define the predecessor function by primitive recursion:

$$\begin{aligned} h(x_1, x_2) &= x_1 \\ P(0) &= 0 \\ P(x+1) &= h(x, P(x)) \end{aligned} \tag{1.10}$$

Functions are great. Their syntax can result in complicated expressions, however. Therefore, the book adopts a more familiar or succinct notation for the reminder of our discussion:

- Since the projection function *selects and returns the $j^{th}$ argument*, we simply make it return $x_j$, instead of $U_j^n(x_1, x_2, \ldots, x_n)$.
- $x + y$ is preferred to $A(x, y)$.
- $x \times y$ is preferred to $M(x, y)$.
- Proper subtraction (the one that never produces a negative number) is shown by the $\dot{-}$ symbol[3]: $x \dot{-} y$.

[3] Macro '\dotdiv' from the `mathabx` package.

# Bibliography

[1] Paul R. Halmos. *I Want to be a Mathematician.* MAA Press, 2005.

[2] Carl H. Smith. *Recursive Introduction to the Theory of Computation.* Springer-Verlag, 1994.